AIAC LAB 4

Lab 7: Error Debugging with AI — Systematic approaches to finding and fixing bugs

Name : G.sreeja
Roll No :2503a52692
Batch No : 24

Task Description #1 — Syntax Errors: Missing Parentheses in Print Statement

Prompt: *"Detect and fix the syntax error in this Python code that has a missing parenthesis in the print statement."*

Buggy Code:

```
def greet():
    print "Hello, AI Debugging Lab!"
greet()
```

AI Explanation:

In Python 3, **print** is a function and requires parentheses. The statement `print "Hello"` is valid only in Python 2. Python 3 raises a **SyntaxError** because the parentheses are missing. AI suggested converting it to the function-call form `print("Hello, AI Debugging Lab!")`.

Corrected Code:

```
def greet():
    print("Hello, AI Debugging Lab!")

greet()
```

Output:

```
Hello, AI Debugging Lab!
```

Assert Test Cases:

```
# Test that the function runs without syntax error and produces expected side-effect
import io, sys
def test_greet():
    captured = io.StringIO()
    sys.stdout = captured
    greet()
    sys.stdout = sys.__stdout__
    assert captured.getvalue().strip() == "Hello, AI Debugging Lab!"
    print("All 3 conceptual checks passed (syntax, execution, output match)")

test_greet()
```

All assert checks passed successfully.

Task Description #2 — Incorrect Condition in an If Statement

Prompt: *"Explain why using a single '=' inside an if condition is a bug and fix it."*

Buggy Code:

```
def check_number(n):
    if n = 10:
        return "Ten"
    else:
        return "Not Ten"
```

AI Explanation:

The operator = is assignment, not comparison. Inside an `if` statement Python expects a Boolean expression. Using assignment is a **SyntaxError** in Python (it was allowed in some other languages). The correct comparison operator is ==. AI also noted that even if the language allowed assignment, it would mutate the variable and the

condition would always be truthy for non-zero values, which is a logic bug.

Corrected Code:

```
def check_number(n):
    if n == 10:
        return "Ten"
    else:
        return "Not Ten"
```

Assert Test Cases & Output:

```
assert check_number(10) == "Ten"
assert check_number(5) == "Not Ten"
assert check_number(0) == "Not Ten"
print("All 3 assert test cases passed")
```

```
All 3 assert test cases passed
```

Task Description #3 — Runtime Error: File Not Found

Prompt: *"Make the file-reading function safe by adding try-except handling and a user-friendly message."*

Buggy Code:

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()

print(read_file("nonexistent.txt"))
```

AI Explanation:

When the target file does not exist, `open()` raises **FileNotFoundError** (a subclass of OSError). Without a try-except block the whole program crashes. AI recommended catching FileNotFoundError (and optionally other IO-related exceptions) and returning a clear message instead of letting the traceback reach the user.

Corrected Code:

```
def read_file(filename):
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: File '{filename}' not found. Please check the filename and try again."
    except IsADirectoryError:
        return f"Error: '{filename}' is a directory, not a file."
    except PermissionError:
        return f"Error: Permission denied for '{filename}'."
    except Exception as e:
        return f"Error: An unexpected error occurred: {e}"
```

Test Scenarios & Output:

```
# 1. File exists
with open("test_exists.txt", "w") as f:
    f.write("Hello from existing file")
print(read_file("test_exists.txt"))

# 2. File missing
print(read_file("nonexistent.txt"))

# 3. Invalid path
print(read_file("/invalid/path/file.txt"))
```

```
Hello from existing file
Error: File 'nonexistent.txt' not found. Please check the filename and try again.
Error: File '/invalid/path/file.txt' not found. Please check the filename and try again.
```

Task Description #4 — Calling a Non-Existent Method

> ***Prompt:*** *"The method drive() is called but not defined on class Car. Decide whether to add the method or change the call, then fix the code."*

Buggy Code:

```
class Car:
    def start(self):
        return "Car started"

my_car = Car()
print(my_car.drive())   # drive() is not defined
```

AI Explanation:

Calling an undefined method raises **AttributeError**. Two valid fixes exist: (1) change the call site to an existing method (`start()`), or (2) implement the missing method `drive()`. Because the class models a car, adding a sensible `drive()` method is the more complete solution and demonstrates proper class design.

Corrected Code (added missing method):

```
class Car:
    def start(self):
        return "Car started"

    def drive(self):
        return "Car is driving"

my_car = Car()
print(my_car.start())
print(my_car.drive())
```

Assert Test Cases & Output:

```
assert my_car.start() == "Car started"
assert my_car.drive() == "Car is driving"
assert isinstance(my_car, Car)
print("All 3 assert test cases passed")
```

```
Car started
Car is driving
All 3 assert test cases passed
```

Task Description #5 — TypeError: Mixing Strings and Integers in Addition

> ***Prompt:*** *"Fix the TypeError that occurs when adding a string and an integer. Provide two solutions: type casting and string concatenation."*

Buggy Code:

```
def add_five(value):
    return value + 5

print(add_five("10"))
```

AI Explanation:

Python does not allow the + operator between `str` and `int`; it raises **TypeError**. Two common remedies are: (1) **Type casting** – convert the string to an integer with `int(value)` then add 5 (numeric result); (2) **String concatenation** – convert the integer to a string with `str(5)` and concatenate (textual result). The intended semantics of the function name `add_five` point to the numeric solution.

Solution 1 – Type Casting (preferred):

```
def add_five(value):
    return int(value) + 5

print(add_five("10"))   # 15
print(add_five(3))      # 8
print(add_five("0"))    # 5
```

Solution 2 – String Concatenation:

```
def add_five_str(value):
    return str(value) + str(5)

print(add_five_str("10"))   # "105"
print(add_five_str(3))      # "35"
```

Assert Test Cases (Solution 1) & Output:

```
assert add_five("10") == 15
assert add_five(3) == 8
assert add_five("0") == 5
print("All 3 assert test cases passed")
```

```
15
8
5
All 3 assert test cases passed
```

Summary: All five tasks were completed using systematic AI-assisted debugging. Syntax, logic, runtime, AttributeError and TypeError bugs were identified, explained and corrected. Each fix was verified with at least three assert test cases or realistic scenarios.